

Похибки й матриці у Python

Лабораторна робота №2

60 хвилин пояснення. Python і NumPy.

Перехід від лабораторної №1

Формули, умови й цикли вже застосовано.

Чистий запуск notebook дає відтворюваний результат.

Тепер перевіряємо точність і форму даних.

Округлення і порівняння наближень

«Точніше» потребує критерію.

Масштаб числа змінює зміст абсолютної похибки.

Порівняємо однакову метрику для обох рівностей.

Практичний контекст: контроль округленого звіту

Результат 0.463 і результат 6.63 мають різний масштаб.

Відсотковий показник дозволяє порівнювати.

Вхідне значення має бути зафіксоване до форматування.

1	$\sqrt{44} = 6,63;$	$\frac{19}{41} = 0,463.$
---	---------------------	--------------------------

Фрагмент завдання 2.1, посібник 6619, с. 37

Визначення: дві міри похибки

$$\Delta = |x - a|, \quad \delta = \frac{|x - a|}{|x|} \ (x \neq 0), \quad p = 100\delta\%$$

Абсолютна: $|x - a|$.

Відносна: $|x - a|/|x|$ для $x \neq 0$.

Відсоткова: $100\delta \%$.

Округлення: цифра 5 і число float

У правилі посібника точну середину округлюють до парної цифри.

`round(x, n)` задає десяткові позиції, не значущі цифри.

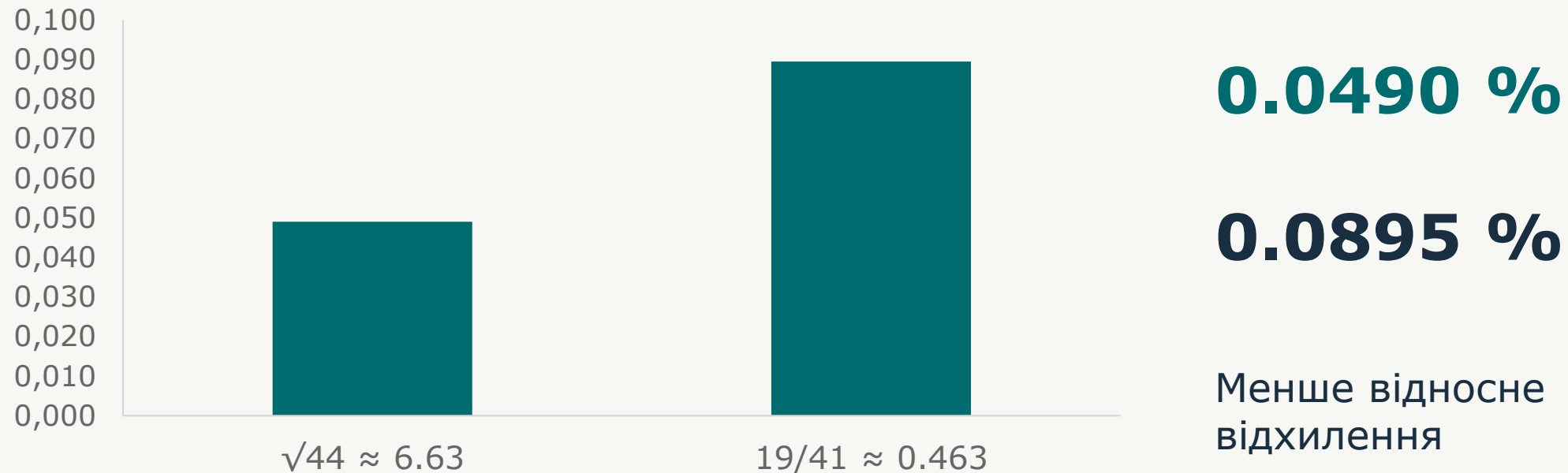
Двійкове float може не представляти десяткову середину точно.

```
print(round(2.5), round(3.5))  
print(round(2.675, 2))
```

Навчальний фрагмент синтаксису

Варіант 1: точність двох рівностей

Відносна похибка, %



Python Challenge: завдання 2.1

Взяти дві рівності свого варіанта зі с. 37.

Обчислити контрольні значення та дві похибки.

Вивести висновок за однаковим відносним критерієм.

Передавання похибки через формулу

Кожен вхід має власну межу похибки.

Формула може посилити або послабити її вплив.

Для висновку потрібна модель поширення похибки.

Практичний контекст: бюджет чутливості

Коефіцієнт чутливості: $\partial f / \partial x_i$.

Внески показують, який вхід найбільше впливає.

Лінійна оцінка потребує перевірки на всій області.

Визначення: оцінка першого порядку

$$L = \sum_{i=1}^n |\partial_i f(\hat{x})| E_i, \quad |f(x) - f(\hat{x})| \leq \sum_{i=1}^n M_i E_i$$

Дані: $|x_i - \hat{x}_i| \leq E_i$.

$L = \sum |\partial_i f(\hat{x})| E_i$ — лінійний внесок.

Для гарантії потрібні межі похідних на всій області та умови гладкості.

Зразок із посібника: степені й корінь

$$x = \frac{m^2 n^3}{\sqrt{k}}, \quad \delta_{\text{lin}} = 2 \frac{E_m}{m} + 3 \frac{E_n}{n} + \frac{E_k}{2k}$$

$x = m^2 n^3 / \sqrt{k}$, усі параметри додатні.

$m = 2.3 \pm 0.002$; $n = 1.45 \pm 0.001$; $k = 0.678 \pm 0.0003$.

У першому порядку $\delta \approx 2E_m/m + 3E_n/n + E_k/(2k)$.

Точний інтервал для додатного зразка

$$x_{\min} = \frac{(m - E_m)^2 (n - E_n)^3}{\sqrt{k + E_k}}, \quad x_{\max} = \frac{(m + E_m)^2 (n + E_n)^3}{\sqrt{k - E_k}}$$

Функція зростає за m, n і спадає за k .

$x_{\min}$: нижні m, n та верхній k .

$x_{\max}$: верхні m, n та нижній k .

Python Challenge: завдання 2.2

$$X = \frac{ab}{\sqrt[3]{c}} \quad (k = 1)$$

Перенести формулу свого варіанта зі с. 38–39.

Перевірити області коренів і ненульові знаменники.

Обчислити центр, лінійну оцінку та незалежну перевірку.

Матриці, розміри та операції

Матриця має форму, а не лише значення.

Розміри визначають допустимі дії.

NumPy іноді виконує іншу операцію з тими самими символами.

Визначення: розмір і транспонування

$$A \in \mathbb{R}^{m \times n}, \quad A^T \in \mathbb{R}^{n \times m}, \quad (A^T)_{ji} = A_{ij}$$

$A \in \mathbb{R}^{m \times n}$ означає m рядків та n стовпців.

$$A^T \in \mathbb{R}^{n \times m}; (A^T)_{ji} = A_{ij}.$$

Для завдання: $A.\text{shape}=(4,4)$, $B.\text{shape}=(4,1)$.

NumPy array: побачити форму перед результатом

`np.array` створює 2D масив із вкладених рядків.

`.shape` повертає (рядки, стовпці).

`dtype` визначає числовий тип елементів.

```
import numpy as np
C = np.array([[2, 5], [7, 11]])
print(C.shape, C.dtype)
print(C.T)
```

Навчальний фрагмент синтаксису

Умова $A+B$: що саме обчислює NumPy

Операція	Умова	Результат
Матрична сума $A+B$	Однакові розміри	Не визначена
NumPy $A+B$	Broadcast (4,1) до (4,4)	Масив (4,4)
Добуток $A@B$	4 внутрішні розміри	Стовпець (4,1)

Ті самі символи задають різні операції.

Визначення: добуток матриць

$$C = AB, \quad c_{ij} = \sum_{k=1}^n a_{ik}b_{kj}, \quad A_{m \times n} B_{n \times p} \in \mathbb{R}^{m \times p}$$

$$A(m,n) \cdot B(n,p) \rightarrow C(m,p).$$

$$c_{ij} = \sum_k a_{ik}b_{kj}.$$

* для ndarray множити поелементно, @ виконує матричний добуток.

Визначення: обернена матриця

$$\det A \neq 0 \iff \exists A^{-1}, \quad AA^{-1} = I$$

$\det(A) \neq 0$ для квадратної A гарантує A^{-1} .

$AA^{-1} = A^{-1}A = I$ в точній арифметиці.

У float перевіряємо залишок, а не рівність бітів.

Степені, скаляр і контроль форми

$A^2 = A @ A$; $A^3 = A @ A @ A$.

$A * x$ масштабує; $-A$ є протилежною матрицею.

$A + x$ у NumPy додає скаляр поелементно.

Python Challenge: завдання 2.3

$\alpha = 0.01k$; створити $A(4,4)$, $B(4,1)$.

$\det$, inv із перевіркою, транспонування, A^2 , A^3 .

$A+x$, $A*x$, $A+B$ з поясненням, $A@B$.

Поелементна функція і перевірка

Функція діє на кожний елемент окремо.

Форма A зберігається.

Область визначення перевіряємо для всіх елементів.

Визначення: поелементне відображення

$$B_{ij} = f(A_{ij}), \quad \text{shape}(B) = \text{shape}(A)$$

$B_{ij} = f(A_{ij}), \text{shape}(B) = \text{shape}(A).$

Для $\sqrt{a}$ потрібно $a \geq 0$; для $\ln a$ потрібно $a > 0$.

$f(A)$ у цьому завданні означає поелементний виклик.

NumPy ufunc: застосування до масиву

`np.sqrt(A)`, `np.cos(A)`, `np.log(A)`
працюють поелементно.

`math.cos` призначена для скалярного аргументу.

Умови функції перевіряємо до виклику.

```
import numpy as np
C = np.array([[0., 1.], [2., 3.]])
D = np.cos(C)
print(C.shape, D.shape)
```

Навчальний фрагмент синтаксису

Варіант 1: корінь і косинус

$$f(a) = \sqrt{a} + \cos a + 0.2a, \quad B_{ij} = f(A_{ij})$$

$$f(a) = \sqrt{a} + \cos(a) + 0.2a.$$

A має 2 рядки та 3 стовпці, усі $a_{ij} > 0$.

Перевірка: $B_{11} = f(0.54)$, $\text{shape}(B) = (2, 3)$.

Python Challenge: завдання 2.4

Взяти функцію й матрицю свого варіанта зі с. 40–42.

Перевірити область для кожної комірки до обчислення.

Обчислити B й незалежно перевірити два елементи.

Звіт і перевірки для чотирьох завдань

Варіант і точний запис постановки.

Проміжні величини, форми масивів, підписані результати.

Незалежні перевірки та чистий запуск notebook.

Результат: точність і зміст операції

Порівнюємо наближення за названим критерієм.

Похибку функції оцінюємо з явними припущеннями.

Форму, добуток і поелементну дію перевіряємо окремо.

Екзаменаційні питання: 9–14.